

Titanic Classification Practical

Logistic Regression, Decision Tree and K-Nearest Neighbors (KNN)

This PDF contains the complete continuation code for the Titanic notebook, designed to be kept with the project when uploading it to GitHub.

Existing preprocessing used

The original notebook loads Titanic-Dataset.csv, fills missing Age values with the median, fills missing Embarked values with the mode, drops Cabin, encodes Sex as 0/1, one-hot encodes Embarked, visualizes age distribution, and saves cleaned_titanic.csv. [filecite](#)turn1file0

Complete Classification Code

```
# =====
# CLASSIFICATION MODELS
# Logistic Regression, Decision Tree and KNN
# =====

# Import required libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# =====
# 1. Prepare Features (X) and Target (y)
# =====

columns_to_drop = ['Survived', 'Name', 'Ticket', 'PassengerId']

X = titanic_data.drop(
    columns=[col for col in columns_to_drop if col in titanic_data.columns]
)

y = titanic_data['Survived']

print("Features used for prediction:")
print(X.columns.tolist())

print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)

# =====
# 2. Train-Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining data size:", X_train.shape)
print("Testing data size:", X_test.shape)

# =====
# 3. Logistic Regression
# =====

logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)

y_pred_logistic = logistic_model.predict(X_test)

logistic_accuracy = accuracy_score(y_test, y_pred_logistic)

print("\n--- Logistic Regression ---")
print("Accuracy:", logistic_accuracy)
print("Accuracy (%):", round(logistic_accuracy * 100, 2), "%")

# =====
# 4. Confusion Matrix
# =====

cm = confusion_matrix(y_test, y_pred_logistic)

print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['Did Not Survive', 'Survived']
)

disp.plot()
plt.title("Logistic Regression - Confusion Matrix")
```

```

plt.show()

# =====
# 5. Decision Tree Classifier
# =====

decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train, y_train)

y_pred_tree = decision_tree.predict(X_test)

tree_accuracy = accuracy_score(y_test, y_pred_tree)

print("\n--- Decision Tree Classifier ---")
print("Accuracy:", tree_accuracy)
print("Accuracy (%)":, round(tree_accuracy * 100, 2), "%")

# =====
# 6. K-Nearest Neighbors (KNN)
# =====

knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

y_pred_knn = knn_model.predict(X_test)

knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("\n--- K-Nearest Neighbors (KNN) ---")
print("Accuracy:", knn_accuracy)
print("Accuracy (%)":, round(knn_accuracy * 100, 2), "%")

# =====
# 7. Compare Model Accuracies
# =====

print("\n=====")
print("MODEL ACCURACY COMPARISON")
print("=====")

print("Logistic Regression :", round(logistic_accuracy * 100, 2), "%")
print("Decision Tree       :", round(tree_accuracy * 100, 2), "%")
print("KNN                  :", round(knn_accuracy * 100, 2), "%")

model_comparison = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Decision Tree',
        'KNN'
    ],
    'Accuracy': [
        logistic_accuracy,
        tree_accuracy,
        knn_accuracy
    ]
})

model_comparison['Accuracy (%)'] = (
    model_comparison['Accuracy'] * 100
).round(2)

print("\nModel Comparison:")
display(model_comparison)

# =====
# 8. Tune max_depth of Decision Tree
# =====

depth_values = [2, 3, 4, 5, 6, 8, 10, None]

depth_results = []

for depth in depth_values:
    model = DecisionTreeClassifier(
        max_depth=depth,
        random_state=42
    )

```

```

model.fit(X_train, y_train)
predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

depth_results.append({
    'max_depth': depth,
    'accuracy': accuracy,
    'accuracy (%)': round(accuracy * 100, 2)
})

depth_results_df = pd.DataFrame(depth_results)

print("\n--- Decision Tree max_depth Tuning ---")
display(depth_results_df)

# =====
# 9. Plot max_depth vs Accuracy
# =====

depth_labels = [
    '2', '3', '4', '5', '6', '8', '10', 'None'
]

plt.figure(figsize=(8, 5))

plt.plot(
    depth_labels,
    depth_results_df['accuracy (%)'],
    marker='o'
)

plt.title("Decision Tree: max_depth vs Accuracy")
plt.xlabel("max_depth")
plt.ylabel("Accuracy (%)")
plt.grid(True)
plt.show()

# =====
# 10. Find Best max_depth
# =====

best_row = depth_results_df.loc[
    depth_results_df['accuracy'].idxmax()
]

print("\n=====")
print("BEST DECISION TREE RESULT")
print("=====")

print("Best max_depth:", best_row['max_depth'])
print("Best Accuracy:", best_row['accuracy (%)'], "%")

# =====
# 11. Final Conclusion
# =====

print("\n=====")
print("CONCLUSION")
print("=====")

print(
    "Logistic Regression Accuracy:",
    round(logistic_accuracy * 100, 2), "%"
)

print(
    "Decision Tree Accuracy:",
    round(tree_accuracy * 100, 2), "%"
)

print(
    "KNN Accuracy:",
    round(knn_accuracy * 100, 2), "%"
)

print(
    "Best Decision Tree max_depth:",
    best_row['max_depth']
)

```

```
print(
    "Best Decision Tree Accuracy:",
    best_row['accuracy (%)'], "%"
)
```

GitHub note

For GitHub, the most useful files are the notebook (.ipynb), dataset/cleaned CSV if permitted, and this PDF as a readable project/reference document.